

FlashAttention: GPU-Efficient Attention

CS290W LLM07 Lecture

LU Yuzhou

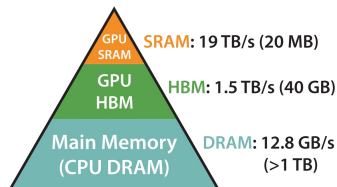
The Problem: Attention is Memory-Bound

- Standard self-attention has $O(N^2)$ HBM memory access
- HBM (High-Bandwidth Memory) = slow off-chip DRAM
- For long sequences, memory bandwidth is the bottleneck, not FLOPs

FlashAttention Solution: Leverage GPU memory hierarchy to minimize HBM access

Key Insight

On-chip SRAM (shared memory) is $\sim 10\times$ faster than HBM. Keep intermediate results on-chip!



图片来源: 论文 FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness

GPU Memory Hierarchy

Level	Size (A100)	Bandwidth	Latency
Registers	256 KB / SM	—	1 cycle
Shared Memory (SRAM)	164 KB / SM	19 TB/s	30 cycles
L2 Cache	40 MB	5 TB/s	200 cycles
HBM (Global)	80 GB	2.0 TB/s	400+ cycles

- Shared memory bandwidth: **19 TB/s** vs HBM: **2 TB/s**
- Goal: Minimize HBM round-trips by keeping intermediates in SRAM

Measuring Actual Memory Bandwidth

Lab Exercise: Measure HBM/GDDR bandwidth on your machine using tensor copy:

```
def measure_memory_bandwidth(device, size_mb=256):  
    num_elements = int(size_mb * 1e6 / 4) # float32  
    src = torch.randn(num_elements, device=device)  
  
    # Benchmark tensor clone (HBM read + write)  
    t0 = time.perf_counter()  
    for _ in range(repeats):  
        _ = src.clone()  
    elapsed = (time.perf_counter() - t0) / repeats  
  
    bandwidth = (num_elements * 4 / elapsed) / 1e9 # GB/s  
    return bandwidth
```

Example Result (RX 7900 XTX)

HBM Bandwidth: **391 GB/s** (measured)

SRAM Bandwidth: **19,000 GB/s** (typical)

→ SRAM is $\sim 49\times$ faster!

Standard Attention IO Analysis: Why $O(N^2)$ HBM Access?

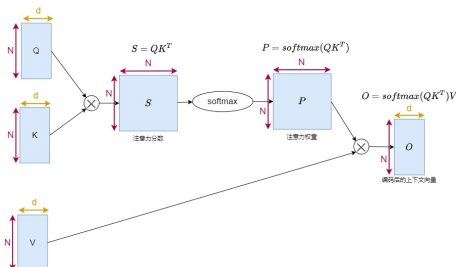
For sequence length N and head dimension d :

- 1 Read Q, K from HBM $\rightarrow$ Compute $S = QK^T \rightarrow$ **Write S to HBM**
- 2 Read $S \rightarrow$ Compute $P = \text{softmax}(S) \rightarrow$ **Write P to HBM**
- 3 Read $P, V \rightarrow$ Compute $O = PV \rightarrow$ Write O to HBM

Total HBM Access

$O(Nd + N^2)$ — dominated by the $N \times N$ matrices S and P

Example: $N = 4096$, FP16 $\rightarrow$ Score matrix = **33.6 MB per head**



Optimization 1 Tiling: Partitioning for SRAM

Tiling partitions matrices into blocks that fit in shared memory.

- Each tile is loaded **once** from HBM
- Reused for multiple computations in SRAM
- Example: 32×32 matmul with 16×16 tiles
- Global memory reads: $65,536 \rightarrow 4,096$ (**$16 \times$ reduction**)

HIP Kernel: Tiled Matrix Multiplication

Real GPU implementation using HIP:

```
--global__ void tiledMatMul(float* A, float* B, float* C) {
    __shared__ float As[TILE][TILE];
    __shared__ float Bs[TILE][TILE];

    int tx = threadIdx.x, ty = threadIdx.y;
    int row = blockIdx.y * TILE + ty;
    int col = blockIdx.x * TILE + tx;
    float sum = 0.0f;

    for (int t = 0; t < numTiles; t++) {
        // Load tiles from HBM to SRAM
        As[ty][tx] = A[row * K + t*TILE + tx];
        Bs[ty][tx] = B[(t*TILE + ty) * N + col];
        __syncthreads(); // Wait for all threads

        // Compute in SRAM (fast!)
        for (int k = 0; k < TILE; k++)
            sum += As[ty][k] * Bs[k][tx];
        __syncthreads();
    }
    C[row * N + col] = sum;
}
```

Normal Softmax: Two-Pass Algorithm

Softmax converts real numbers to a probability distribution:

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_{j=1}^N e^{x_j}}$$

Numerical Stability: Subtract max to prevent overflow:

$$\text{softmax}(x)_i = \frac{e^{x_i - m}}{\sum_{j=1}^N e^{x_j - m}}, \quad m = \max_j x_j$$

Two-Pass Algorithm

Pass 1 (REDUCE): Find $m = \max(x)$, compute $s = \sum e^{x_j - m}$

Pass 2 (MAP): For each i : $p_i = e^{x_i - m} / s$

Softmax Resource Analysis

Step	Operation	Type	Resource
Pass 1, Step 1	$m = \max(x)$	REDUCE	Memory-bound
Pass 1, Step 2	$s = \sum e^{x_j - m}$	REDUCE	Memory-bound
Pass 2, Step 3	$p_i = e^{x_i - m} / s$	MAP	Compute-bound

Key Observations:

- **REDUCE operations** are memory-bound: must read entire input from HBM
- **MAP operation** is compute-bound: exponential + division per element
- Total HBM traffic: $O(N)$ per row $\rightarrow O(N^2)$ **for full attention matrix**

Problem

Standard softmax must **materialize full $N \times N$ matrix** in HBM

For $N = 4096$: 33.6 MB per head — too large for SRAM!

Online Softmax: Single-Pass Solution

Challenge for Attention

Row-wise softmax depends on the **entire row**. Cannot compute softmax per-tile until all tiles are seen.

Online softmax maintains running statistics (m, d) updated incrementally:

Property	Standard	Online
Passes	2 (must see all data)	1 (streaming)
State	Full input stored	Only (m, d) — 2 values!
Use Case	Small vectors	Large matrices (attention)

Update Formulas:

$$m_{\text{new}} = \max(m_{\text{old}}, m_{\text{block}})$$
$$d_{\text{new}} = d_{\text{old}} \cdot e^{m_{\text{old}} - m_{\text{new}}} + \sum_{j \in \text{block}} e^{x_j - m_{\text{new}}}$$

FlashAttention V1: Core Algorithm

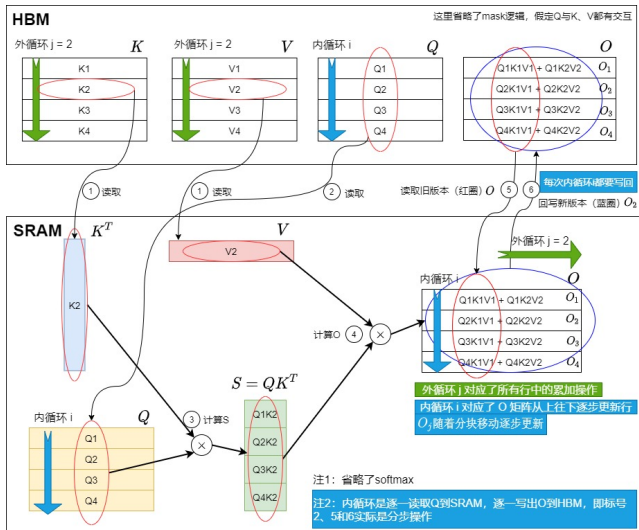
Process Q, K, V in **tiles**:

- 1 Divide K, V into blocks of B_c rows; Q into blocks of B_r rows
- 2 For each Q-block, iterate over K/V-blocks:
 - Load Q-tile, K-tile, V-tile into SRAM
 - Compute local $S_{\text{tile}} = Q_{\text{tile}} K_{\text{tile}}^T$
 - Update running softmax (online softmax)
 - Accumulate output: $O += \text{softmax_tile} \times V_{\text{tile}}$
- 3 Final rescaling after all blocks

HBM Access Complexity

$O(N^2 d^2 / M)$ where $M = \text{SRAM size}$ — much less than $O(Nd + N^2)$

FlashAttention V1: Diagram



FlashAttention V1 vs V2: Why V2 is Faster

V1 Pseudocode (K/V-outer):

```
for j in K/V_blocks:           # Outer: K/V
    for i in Q_blocks:         # Inner: Q
        S_tile = Q[i] @ K[j].T
        m_new = max(m[i], m_tile)
        O[i] *= exp(m[i] - m_new) # RESCALE EVERY ITER!
        O[i] += exp(S_tile) @ V[j]
```

V2 Pseudocode (Q-outer):

```
for i in Q_blocks:             # Outer: Q
    for j in K/V_blocks:       # Inner: K/V
        S_tile = Q[i] @ K[j].T
        m_new = max(m[i], m_tile)
        O[i] += exp(S_tile) @ V[j] # NO RESCALE!
    O[i] /= l[i]                # FINAL RESCALE ONLY
```

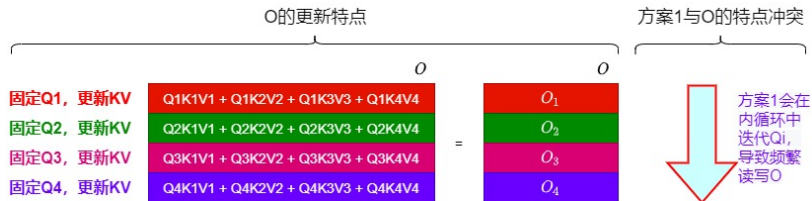
FlashAttention V1 vs V2: Why V2 is Faster? (Continued)

V2 Advantages

Deferred rescaling: $O(N)$ writes vs $O(N^2)$

Better locality: Each SM owns Q-block, processes all K/V

No inter-warp sync: Q-split instead of K-split



IO Complexity Comparison

Method	HBM Reads/Writes	Peak HBM
Standard Attention	$O(Nd + N^2)$	$O(N^2)$
FlashAttention V1	$O(N^2 d^2 / M)$	$O(N)$
FlashAttention V2	Same, fewer ops	$O(N)$

Where:

- M = SRAM size per SM (100-200 KB)
- N = sequence length
- d = head dimension (typically 64-128)

Key Takeaway

For typical settings ($d = 64-128$, $M \gg d^2$), FlashAttention eliminates **almost all N^2 -sized HBM traffic**

Memory Savings: Real Numbers

Intermediate memory per head (FP16, $d = 128$):

N	Standard	Flash	Savings
512	0.52 MB	0.0008 MB	655×
1024	2.10 MB	0.0016 MB	1311×
2048	8.39 MB	0.0031 MB	2687×
4096	33.55 MB	0.0063 MB	5338×
8192	134.22 MB	0.0125 MB	10701×

Why This Matters

Standard attention runs out of memory for long sequences.

FlashAttention enables **context lengths of 64K+** on consumer GPUs.

Using FlashAttention in PyTorch

PyTorch $\geq$ 2.0 provides automatic FlashAttention via SDPA:

```
import torch.nn.functional as F

# SDPA automatically uses FlashAttention when available
output = F.scaled_dot_product_attention(Q, K, V)
```

- `scaled_dot_product_attention` (SDPA) selects backend:
 - FlashAttention (ROCm/CUDA)
 - Memory-Efficient Attention
 - Math fallback
- No code changes needed!

Dedicated Library

```
pip install flash-attn → flash_attn_func(Q, K, V)
```

Benchmark: Standard vs Flash Attention

Typical speedups on modern GPUs (A100, H100, RX 7900 XTX):

N	Standard	SDPA (Flash)	Speedup
256	0.15 ms	0.12 ms	1.25×
512	0.45 ms	0.28 ms	1.61×
1024	1.52 ms	0.78 ms	1.95×
2048	5.89 ms	2.45 ms	2.40×
4096	23.45 ms	7.82 ms	3.00×

- Speedup increases with sequence length
- Memory savings: **2–5×** lower peak HBM usage

Summary: Key Takeaways

- ① **GPU Memory Hierarchy:** SRAM is $10\times$ faster than HBM
- ② **Tiling:** Partition computation to fit in SRAM
- ③ **Softmax Analysis:** REDUCE is memory-bound, MAP is compute-bound
- ④ **Online Softmax:** Single-pass with running stats (m, d)
- ⑤ **FlashAttention V1:** $O(N^2 d^2 / M)$ HBM vs $O(N^2)$ standard
- ⑥ **FlashAttention V2:** Deferred rescaling, Q-outer loop, better warp usage

Bottom Line

FlashAttention is not about reducing FLOPs — it's about **reducing memory traffic**.
This makes it **2–4× faster** and enables **longer context lengths**.

Experiment Further

- Install flash-attn package: `pip install flash-attn`
- Compare FP16 vs BF16 performance
- Benchmark causal masks (autoregressive) vs bidirectional
- Profile with `torch.profiler` to see SRAM/HBM usage
- Read the papers:
 - FlashAttention V1: [arXiv:2205.14135](https://arxiv.org/abs/2205.14135)
 - FlashAttention V2: [arXiv:2307.08691](https://arxiv.org/abs/2307.08691)

Lab Notebook

See `LLM07-FlashAttention.ipynb` for hands-on exercises:

- Measure memory bandwidth
- Implement tiled matmul (Python + HIP)
- Implement online softmax
- Simulate FlashAttention V2
- Benchmark standard vs SDPA